

EEET2588

Real-time Systems Engineering & Design

Lecture 3: Data Protection

- Theory
- Mutual Exclusion

Dr Linh Tran

Course note acknowledgment: Original course notes developed by Roy Ferguson and Samuel Ippolito from RMIT Melbourne

Lecture 3 Overview

- Lecture 2 and Lab Schedule review
- Simple Producer-Consumer Problem
 - Basic Thread Synchronization
 - Critical Section (CS)
 - Mutual Exclusion
 - Race Condition
 - Deadlock, Starvation and Livelock
- Introduction to QNX Synchronization Primitives

Note: Information on the QNX functions can be found by typing name in to the search box in the following link: <https://www.qnx.com/developers/docs/7.0.0/>

Useful References: [QNX_Neutrino_RTOS_System_Architecture.pdf](#)
[Get_Programming_with_the_QNX_Neutrino_RTOS.pdf](#)

Laboratory program

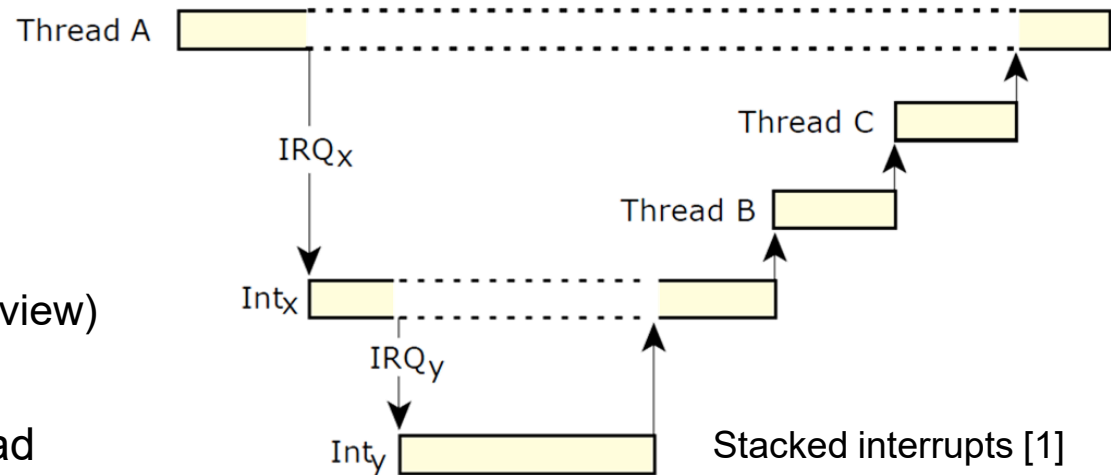
This lecture material directly supports your week 4 lab exercise. It covers theory behind Mutexes, Read/Write Locks, Semaphores, Barriers and Conditional Variables.



Week	Laboratories
1	Getting Started with QNX Tools
2	Ex1 – Intro to QNX and Momentics
3	Ex2 – Threads & Queues
4	Ex3 – Shared memory and Mutual Exclusion
5	Ex4 – State Machines with threads
6	Ex5 – Message Passing between process and nodes
7	
8	Project Work
9	Project Work
10	Project Work
11	Project Work
12	Project Demonstration

Lecture 2 Review

- Process and Threads
 - Process states
 - Thread Preemption
 - Context switching
 - Process switching (mini review)
 - Passing data to a Thread
 - Returning data from a Thread
 - Pass by Reference / Return Address
 - Joined and Detached Threads
 - `pthread_timedjoined()`
 - Changing and Retrieving Thread Attributes (ie. scheduling parameters)
- **Errno** (checking that functions successfully execute correctly)
- Interprocess Communication (IPC)
 - POSIX Message Queues
 - QNX Message Passing
 - Qnet (inbuilt networking)
 - Named connections (ND/PID/CHID)
 - Unnamed connections using Channels (PID/CHID)



The need for data protection

Thread Preemption

• When is a thread preempted?

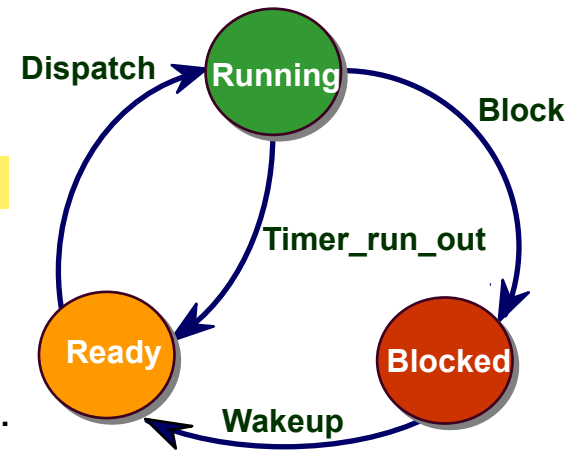
- The running thread is preempted when a high-priority thread is placed on the ready queue (it becomes **READY** as the result of its block condition being resolved).
- The preempted thread is put at the beginning of the ready queue for the that priority, and the higher priority thread runs.

• When is a thread yielded?

- The thread voluntarily yields the CPU by evoking functions like `sched_yield()` or `sleep()`, resulting in it being placed at the end of the ready queue for that priority.
 - Then the highest priority thread runs (which may still be the read thread that just yielded).

• When is a thread blocked?

- The running thread is blocked when it must wait for some event to occur
 - Respond to an IPC request (eg. Message or signal of some sort).
 - Wait on a mutual exclusion operation or a sleep (the latter being a signal).
- The blocked thread is removed from the running array and the highest priority ready thread is then run.
- When a blocked thread is subsequently unblocked, it is placed on the end of the ready queue for the that priority.
- A running thread is temporarily suspended whenever the microkernel is entered as a result of a kernel call, exception or hardware interrupt.

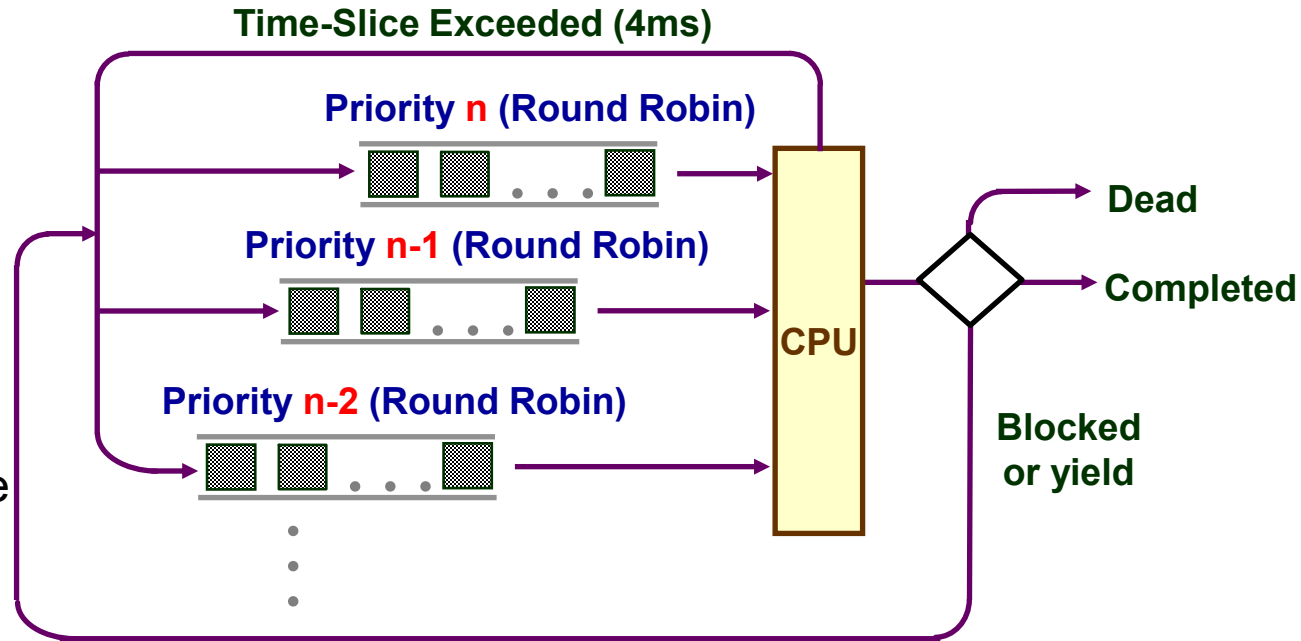


Process Switch - Thread Preemption

In all cases, the highest priority queue has access to CPU resources

Round Robin

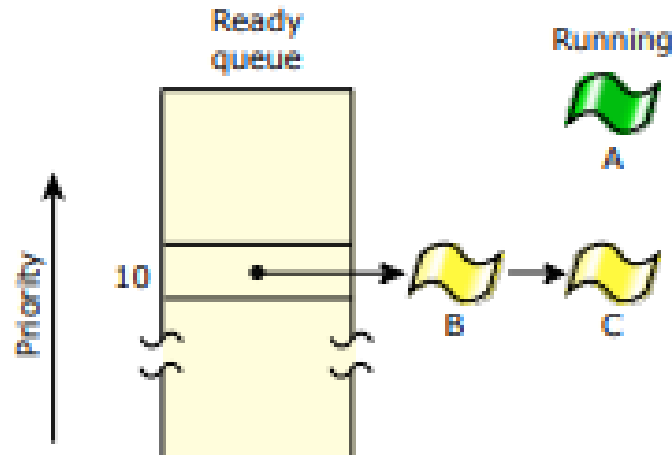
A task will run for a maximum of 4ms before going to back of the queue it came from.



FIFO

Once a task is running it will run to completion.

In both cases, when a task blocks or yield it will go to the back of its priority queue.

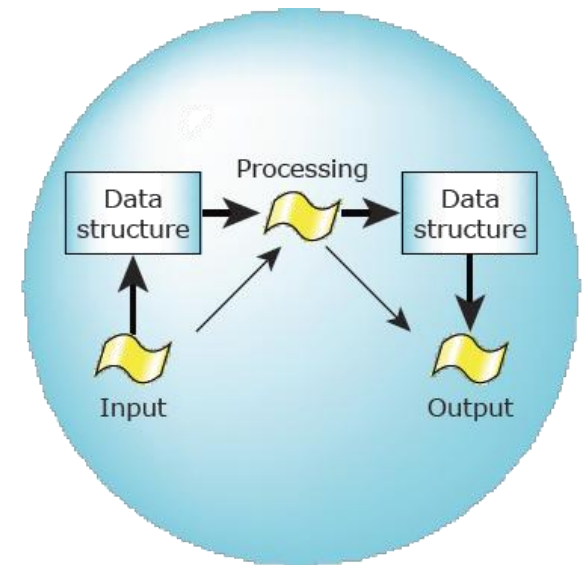


Concurrent Threads and Interrupt Service Routines (ISRs)

- Threads and Interrupt Service Routines (ISRs) *can* and *do* run **concurrently!**
 - We don't know when a **Context switch** or **Process Switches** might occur (even on a single CPU), even for atomic (small) operations such as: ++, --, |=, &=
 - There is no way to control the execution order
 - Relying on thread or ISR prioritization does not work!!
 - Relying on things like FIFO scheduling for synchronization does not work!!
- **Threads and Interrupt Service Routines (ISRs) need protection from themselves and each other**
 - Threads require protection from other Threads and from all ISRs
 - Additionally, an ISR requires protection from other ISRs and all Threads
- Such issues are low-probability events that don't always occur during the coding/development phase, but rather during testing, demos, or out in the field.
- → There is need to minimize or avoid this potential problem.

Single Process with Multiple Threads

- Lastly we can use one process with three threads which share data areas implicitly.
 - Thick lines indicate data flow
 - Control information is used to synchronize the process of allowing the threads to access the shared data.
 - Can be implemented using
 - Message passing, or by using
 - Dedicated synchronization methods such as: **mutexes**, **barriers**, or **semaphores**
- The most tightly-coupled design
- **Disadvantages:** A single process cannot be distributed over a network. Also all components within the processes rely on each other much more.
- **Advantages:** Even though this design is the most inflexible, it is most likely going to be the fastest.
 - There is no thread-to-thread context switches
 - Can use faster synchronization methods (at kernel level)
 - The availability of all threads is known at runtime
 - Limits errors with resources not being available
 - Will have smallest footprint - minimizes data copies



Multiple operations, multiple threads [2]

Producer-Consumer problem

The Producer-Consumer Problem

- A **producer** process produces data that is consumed by a **consumer** process
 - The Produce-Consumer model is commonly used in Device Control or within an OS
 - Especially for handling different I/O devices
 - Typically the producer and consumer operate at different and varying rates
 - but over time a consumer generally consumes faster than the producer produces
 - Input from a keyboard is used up very quickly by a consumer process such as a word processor
- At anytime, either the **producer** or **consumer** process may driven by an **ISR**
 - e.g. A program generating printable output produces characters that are consumed by the printer driver, or display
- To allow the producer and consumer processes to run concurrently, a pool of buffers is typically used that are filled by the producer and emptied by the consumer
 - The **unbounded-buffer** problem allows an unlimited number of buffers, so there is always an empty one for the producer to fill (**does not exist in reality**)
 - The **bounded-buffer** problem assigns a fixed number of buffers so if all are full, the producer must wait (or do something else) until there is a free buffer
 - In either case, the consumer must wait if all buffers are empty

Simple Producer-Consumer Example (Single Buffer - Non Concurrent)

- Lets look at a simple Producer-Consumer process.
- `main()` calls a `producer()` function and then calls the `consumer()` to use the data
 - The `while` loop in the `main()` make the process **cyclical**.
 - This type of software structure is inefficient, as it block the whole process until a character is produced. The `consumer` or `main` functions can't do work during this time.



```
volatile char data = 0;

void producer()
{
    printf("Producer Started...\n");
    int key = 0;

    while(1)
    {
        printf("press a key:\n");
        key = getchar();          // Wait here
        getchar(); // waste carriage return

        data = key;
        break; // need to give control
               // to the consumer
    }
}
```



```
void consumer()
{
    printf("Consumer Started...\n");

    // do something with the data

    printf("Consumer received: %c\n", data);
}
```

```
void main( void )
{
    while(1)
    {
        producer(); // run the producer
        consumer(); // run the consumer
    }
    printf("Never going Terminate...\n");
}
```

Producer-Consumer Example using Concurrent Threads

- This example of the Producer-Consumer process has 3 threads in total.
 - `main` thread, `producer` thread, `consumer` thread
 - The `consumer` thread check for `data != 0`

It appears to work, but what happens if we want to send the value 0 (NULL) between the threads? A solution could be to add a `Flag` variable, but checking a `Flag` and updating the data variable is **NOT** an **atomic operation** and could lead to a **race**

```
volatile char data = 0;

void *producer_ex(void *notused)
{
    printf("Producer Started...\n");
    int key = 0;

    while(1)
    {
        printf("press a key:\n");
        key = getchar();      // Block here
        getchar(); // waste carriage return

        data = key;
        // no need to break here as we
        // want this thread to continue
        // running
    }
}
```

```
void *consumer_ex(void *notused)
{
    printf("Consumer Started...\n");
    while(1) {
        if(data != 0) { // wait here
            printf("Consumer received: %c\n", data);
            data = 0;    // reset data value
        }
    }
}
```

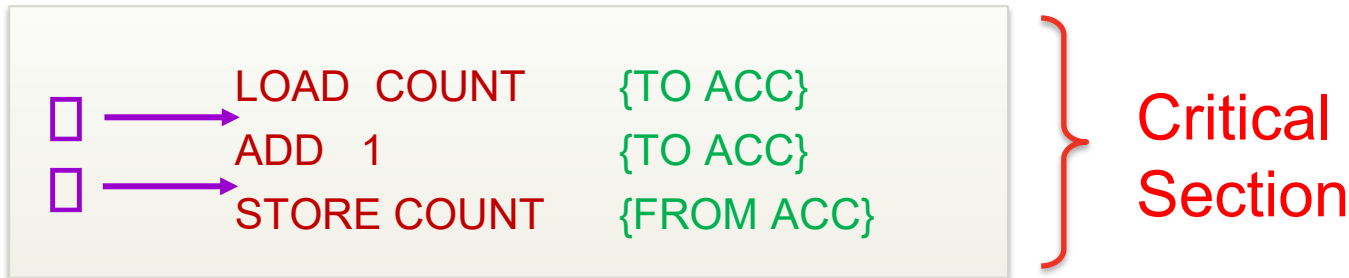
```
void main( void )
{
    pthread_t th1, th2;
    pthread_create(&th1, NULL, consumer_ex, NULL);
    pthread_create(&th2, NULL, producer_ex, NULL);
    pthread_join(th1, NULL); //needed to stay alive
    printf("Never going to Terminate...\n");
}
```

Avoiding a Race Condition

- Adding a **Flag** variable will require the **producer** thread to **set it** and the **consumer** to **clear it**, whilst at the same time both threads need to hand the actual data value.
 - Unfortunately there is no guaranteed way to check a variable and update another without the possibility that a Context or Process Switch occurs in-between – unsafe!!
 - The final result will depend heavily on the order of execution.
 - **The Bank Account Analogy**: A husband and a wife accessing the same account simultaneously from different ATMs – *Which one is presented with truthful information?*
- Basically, if one thread is part way through the updating process when another task access the shared variable/s, a major issue could occur – These types of bugs are called **race conditions!** Some are **very bad** and some are **completely acceptable**.
- Race condition often produce different results every time they occur
 - It might work fine one time; the next time it might crash
 - There is no (practical) way to make a system schedule the thread in exactly the same way it did prior.
- To guarantee that data value will be ready for the **consumer** thread in our **Producer-Consumer Example** we need to prevent the **consumer** thread preempting the **producer** until data has been assigned to the **data** variable and the flag set.
 - We need a method to block each thread when we perform these critical procedures.

Non-atomic Operations and Critical Section

- The section of code when a process needs exclusive access to a resource is known as a **critical section**. In this case the CPU is required exclusively:
 - Consider shared variable update by concurrent processes
 - e.g. `count = count + 1;` or `count++;`
- Compile down to assembly level (e.g. on a RISC CPU) it would look something like:



Consider if an interrupt and context switch to another process that shared the variable **COUNT** occurred at either □ symbol location. This could compromise the value of **COUNT** (i.e. could lose or gain counts)

Shared Resource Update Problem

Take a counter which keeps track of a simple producer and consumer process

Producer: $\text{count} = \text{count} + 1;$

Consumer: $\text{count} = \text{count} - 1;$

If run sequentially the CPU time would look like:

```
T0:Producer LOAD COUNT  //ACC = 5
T1:Producer ADD 1        //ACC = 6
T2:Producer STORE COUNT //COUNT = 6
```

```
T3:Consumer LOAD COUNT  //ACC = 6
T4:Consumer SUB 1       //ACC = 5
T5:Consumer STORE COUNT //COUNT = 5
```

- If run concurrently without protection, COUNT could be corrupted (**worst case**):

```
T0:Producer LOAD COUNT  //ACC = 5
T1:Producer ADD 1        //ACC = 6
T2:Consumer LOAD COUNT  //ACC = 5
T3:Consumer SUB 1       //ACC = 4
T4:Consumer STORE COUNT //COUNT = 4
T5:Producer STORE COUNT //COUNT = 6
```

The correct result ($\text{COUNT} = 5$) occurs only if producer and consumer each execute their sequence of instructions modifying count separately.

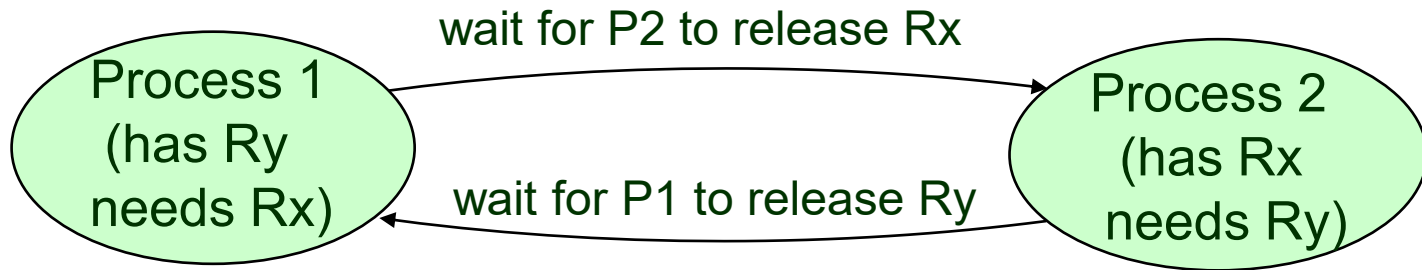
i.e. They have exclusive access to count, which is not a good situation! Results in errors!

Critical Section

- As shown from the previous example, a process can potentially have many **critical sections**. A critical section occurs:
 - Anytime a shared resource is accessed.
 - **This includes every time a shared variable is accessed or used.**
- Traditional OS protect critical sections by using **Mutual Exclusion** operations
 - A **Mutual Exclusion** is a special variable or abstract data type that is used for controlling or monitoring access to a shared resource.
 - Can be a special type of counter (**semaphore**) that is changed (ie. incremented or decremented, or toggled) depending on programmer-defined conditions.
 - A counter value which is non-negative, but more importantly,
 - The OS **guarantees** the checking or modifying the value of semaphore can be done safely, **without creating a race condition** and in an exclusive manner.
 - Incorrect application/design could lead to a RTS suffering from priority inversion.
- **Bathroom Analogy**: A key (token) is used to access a unisex bathroom. If the key is available, you can take it off the hook and lock the bathroom door behind you. When you finish, you unlock the door and place the key on the hook.
 - Everyone else was locked out while you had the key and were locked in.
 - No one was able to open the door while you are in the bathroom.
 - Due to the above, the bathroom could be considered a protected resource.

Deadlock, Starvation and Livelock

- Processes must not be indefinitely postponed from entering their critical section
 - Indefinite Postponement is also known as starvation or livelock
- One or more processes waiting for an event which will not occur is known as deadlock
 - Classic case is the two process circular wait



- Formal definitions:**
 - Deadlock:** Occurs when two or more competing actions are each waiting for the other to finish, and thus neither ever does.
 - Starvation:** Occurs when a process is perpetually denied necessary resources to process its work.
 - Livelock:** A special case of starvation that occurs when two or more processes continually change their state in response to changes in the other processes. The result is that none of the processes will complete. (Can happen if all process do the same thing to avoid deadlock, see: <https://en.wikipedia.org/wiki/Deadlock>)

Mutual Exclusion

Mutual Exclusion

- In order for a process to achieve **exclusive access** to the **critical section** we need to add some extra code before and after the critical section.
- If a *flag* variable is used, then it might also need to be set or cleared as part of the critical section. For example our **producer_ex()** thread could look like:

```
volatile char data = 0;           // Data
volatile int data_ready = 0;     // Flag

void *producer_ex(void *notused)
{
    printf("Producer Started...\n");
    int key = 0;
    while(1)
    {
        printf("press a key:\n");
        key = getchar();
        getchar(); // waste carriage return

        data = key;           // set the data
        data_ready = 1;      // set the flag

    }
}
```

- If we can't get access to the critical section we may be experiencing an **indefinitely postponement event**

**Critical
Section**



```
data = key;           // set the data
data_ready = 1;      // set the flag
```

Consumer thread reads this

Consumer thread clears this

Code required to enforce mutual exclusion is NOT shown

Mutual Exclusion – method 1

- Only one process is allowed to exclusively access a shared resource at one time
 - Required for safety
- But must release the resource within a reasonable time
 - Required for liveness
- The section of code (time) when process has exclusive access to a resource is known as a critical section

// Attempt to enforce mutual exclusion (not real code)
 // for two concurrent processes and maintain liveness

```
int  p1inside = 0;
int  p2inside = 0;

void * process1( void )
{
  while(1)
  {
    while(p2inside)
      { /*wait*/ }
    p1inside = 1;
    criticalsection1;
    p1inside = 0;
    // other process1 code ...
  }
}
```

```
void * process2( void )
{
  while(1)
  {
    while(p1inside)
      { /*wait*/ }
    p2inside = 1;
    criticalsection2;
    p2inside = 0;
    // other process2 code ...
  }
}
```

Generalized
code statement

- This solution could result in a deadlock or livelock if run on a multi-core CPU due to the possibility of the while loops executing at exactly the same time.

Mutual Exclusion – method 2

- What happens if we change the code so that the value of the flag is changed before we test the flag (using the while loop)?

// Second attempt to enforce mutual exclusion
// for two concurrent processes and maintain liveness

```
int p1toenter = 0;
int p2toenter = 0;

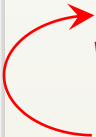
void * process1( void )
{
    while(1)
    {
        p1toenter = 1; //change flag
        while(p2toenter)
            { /*wait*/ }

        criticalsection1;
        p1toenter = 0;
        // other process1 code ...
    }
}
```



```
void * process2( void )
{
    while(1)
    {
        p2toenter = 1; //change flag
        while(p1toenter)
            { /*wait*/ }

        criticalsection2;
        p2toenter = 0;
        // other process2 code ...
    }
}
```



- What happens if we run these process at the same time (multi-core CPU)?
They will set the flags and enter the critical section and corrupt each others data.
This is clearly a Safety issue and will effect the Liveness of the system.

The Path to Mutual Exclusion....

- Text books discuss many, many different type of potential solution to avoid non-mutual exclusive access.
 - Random delays
 - Set and bit check process with random delays, etc.
- Unfortunately there will always be a low-probability that the random delay will be the same value for both processes, or the check/set bit sequence will magically line up
 - This is not mission critical safe
 - We want to avoid fate probability
 - 1 in a million event with CPU operating in the GHz could very well occur.

First real (software based) solutions came in 1965 and 1981:

Dekker's Algorithm is the first known correct solution to the mutual exclusion problem in concurrent programming.

- It allows two threads to share a single-use resource without conflict, using only shared memory for communication.

The Peterson's Algorithm is a simpler version of Dekker's which was developed in 1981. It allows a concurrent programming for mutual exclusion for two threads to share a single-use resource without conflict, using only shared memory for communication.

- Does not require any special OS support! - Avoids deadlock and livelock!

Peterson's Algorithm

- Similar to the previous described methods (method 1 and 2), Peterson's algorithm also uses flag variables but also has an additional shared variable called **turn**.
 - The **turn** variable is used to help insure safety.
 - Process 1 and process 2 can never be in the critical section at the same time.
- Entrance to the critical section is granted for process 1 if process 2 does not want to enter its critical section or if process 2 has given priority to process 1 by setting **turn** to 1.

```
int p1toenter = 0;
int p2toenter = 0;
int turn = 1;    // Favored process

void * process1( void )
{
    while(1)
    {
        p1toenter = 1; // change flag
        turn = 2;      // set for next cycle
        while(p2toenter && turn == 2)
            { /*wait*/ }
        criticalsection1;
        p1toenter = 0;
        // other process1 code ...;
    }
}
```

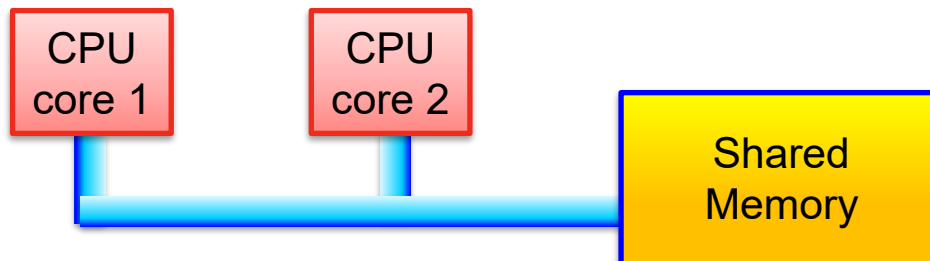
If p2toenter is true: Then we can assume process2 is either about to enter its CS or it is in the CS

```
void * process2( void )
{
    while(1)
    {
        p2toenter = 1; //change flag
        turn = 1;      // set for next cycle
        while(p1toenter && turn == 1)
            { /*wait*/ }
        criticalsection2;
        p2toenter = 0;
        // other process2 code ...;
    }
}
```

Only the code required to enforce mutual exclusion is shown

Peterson's Algorithm

- In order to ensure that the algorithm works, we need to ensure that the thread can enter the Critical Section (CS) under **the following cases / scenarios**:
 1. Only 1 process is active: Process 1 wants to enter CS (simplest case)
 2. Both processes are active: Process 2 wants to enter CS
 3. Both processes are active: Process 2 in CS and Process 1 wants to enter CS
 4. Both processes are active: Process 1 in CS and Process 2 wants to enter CS
 5. Both processes are active: Process 1 and Process 2 want to enter CS at exactly the same time (ie multi-core CPU system)
 - What happens in this situation? **There is a Race to set the turn variable which can only ever be a discrete value (ie. Integer) at any one time.**
 6. Both processes are active: Process 1 wants to enter CS and Process 2 wants to re-enter the CS (**needs to be shown to determine if the algorithm avoids Livelock**)
 - Basically each process should take turn, even in the case where a process switch occurs where one process is the CS on a single CPU system.



Bus arbitration hardware plays an important part in making sure that only 1 CPU core can access a memory location at a time – this ensures the race conditions are mitigated.

Note: Overlapping execution (pipelines), read / write buffers and multi level caches create further complications - Hardware wait states solve this.

Extending Peterson's Algorithm for more than 2 Threads

- As a software only solution, the Peterson's Algorithm can manage 2 processes effectively by using 3 shared memory variables (ie. `p1toenter`, `p2toenter`, `turn`).
 - Similar algorithms such as Peterson's Algorithm are commonly used by OS's for handling parallel processing operations.
 - Can be used for true concurrent processes, although a benign race condition exists.
 - The inner while loops of each process can never both be true - enforces mutual exclusion
- Combined with hardware the Peterson's algorithm can be extended to handle more than 2 processes, however extensions to make it **safely** work for more than 2 process is not trivial
 - Requires recursive behavior (at the software and hardware level)
 - Big increase in complexity
 - Starts to heavily rely on hardware (logic) due to the complexity of modern CPUs (ie, pipelining, caches, etc.) in order to achieve reliable behavior
- When working at the hardware level, Peterson's algorithm is typically not needed to achieve atomic access if being used to provide mutual exclusion for more than 2 processes.
 - Some processors have special instructions, like test-and-set or compare-and-swap, that, by locking the memory bus, can be used to provide mutual exclusion in multi-core systems.

QNX Mutual Exclusion Object

- So far all the solution we have discussed for providing mutual exclusion between two processes has involved testing and setting 3 or more variables
- **QNX Solution:** Protect the critical sections of the code by using a **mutex object**
 - Mutual exclusion, or mutex, is QNX's simplest synchronization service.
 - A mutex is used to ensure exclusive access to data shared between threads.
- A mutex enables you to safely block other threads from accessing a shared resource within a critical data section of code until they are released.

```
pthread_mutex_t mutex =  
    PTHREAD_MUTEX_INITIALIZER;  
  
void * process1( void )  
{  
    while(1)  
    {  
        pthread_mutex_lock(&mutex);  
  
        criticalsection1;  
  
        pthread_mutex_unlock(&mutex);  
  
        // other process1 code ...;  
    }  
}
```

```
void * process2( void )  
{  
    while(1)  
    {  
        pthread_mutex_lock(&mutex);  
  
        criticalsection2;  
  
        pthread_mutex_unlock(&mutex);  
  
        // other process2 code ...;  
    }  
}
```

Producer-Consumer Example using Mutual Exclusion

- In our produce-consumer example, a **QNX mutex** object can be used to protect the **data** and **data_ready** variables in both the **producer** and **consumer** threads.
 - By checking the **mutex** object a thread can determine if the shared memory resource is free for use or update.
 - If its not free the thread blocks instead of burning CPU cycles.

```
volatile char data = 0;           // Data
volatile int data_ready = 0;     // Flag

pthread_mutex_t mutex =
    PTHREAD_MUTEX_INITIALIZER;

void *producer_ex(void *notused)
{
    printf("Producer Started...\n");
    int key = 0;
    while(1)
    {
        printf("press a key:\n");
        key = getchar();
        getchar(); // waste carriage return
        pthread_mutex_lock(&mutex);
        data = key;
        data_ready = 1;
        pthread_mutex_unlock(&mutex);
    }
}
```

```
void *consumer_ex(void *notused)
{
    printf("Consumer Started...\n");
    while(1){
        pthread_mutex_lock(&mutex);
        if(data_ready==1) {
            printf("Consumer received: %c\n", data);
            data_ready = 0;
        }
        pthread_mutex_unlock(&mutex);
    }
}
```

```
void main( void ) {
    pthread_t th1, th2;
    pthread_create(&th1, NULL, consumer_ex, NULL);
    pthread_create(&th2, NULL, producer_ex, NULL);
    pthread_join (th1, NULL);
    printf("Main Terminated...\n");
}
```

Other Useful QNX Synchronization Primitives

- QNX Neutrino RTOS provides the POSIX-standard thread-level synchronization primitives, some of which are useful even between threads in different processes.
 - These operations enable you to safely block other process from accessing critical data section until they are released, without causing your programs to burn through CPU cycles while waiting for a resource to be freed up.
- The [QNX_Neutrino_RTOS_System_Architecture.pdf](#) [1] has a good summary of the available synchronization services (starting at **page 46**).

Used in Lab 3	Synchronization service		Supported between processes	Supported across a QNX Neutrino LAN
	Mutexes	(p. 47)	Yes (with shared mem)	No
	Condvars	(p. 48)	Yes (with shared mem)	No
	Barriers	(p. 49)	No	No
	Sleepon locks	(p. 51)	No	No
	Reader/writer locks	(p. 51)	Yes	No
	Semaphores	(p. 52)	Yes	Yes (named)
	FIFO scheduling	(p. 53)	Yes	No
	Send/Receive/Reply	(p. 53)	Yes	Yes (named)
	Atomic operations	(p. 53)	Yes	No

Real-Time Boeing 787 Dreamliner Issue



- Operators of Boeing 787 Dreamliners are being told to shut down the plane's electrical power periodically after Boeing discovered a software error that could result in a total loss of power.
- Plane's power control units could shut down power generators if they were powered without interruption for 248 days, or about eight months.
- Problem had occurred only in lab simulation – Not on a flying plane...yet
- There are 264 planes effected: including Jetstar, Japan Airlines, Air India and Ethiopian Airlines fleets

[Source: <https://lockerdome.com/smh/7644014082003988>]

References

Images and content used in this presentation can be found:

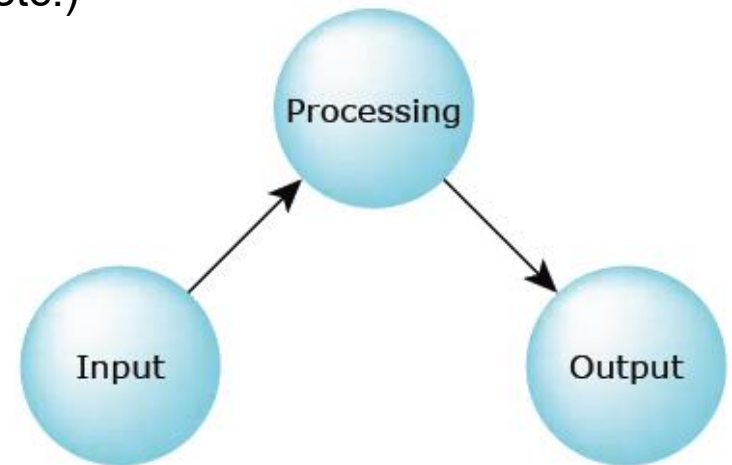
- [1] QNX 700 Documentation - QNX_Neutrino_RTOS_System_Architecture.pdf
- [2] QNX 700 Documentation - Get_Programming_with_the_QNX_Neutrino_RTOS.pdf
- [3] <http://stackoverflow.com/questions/1385843/simple-deadlock-examples>

Acknowledgment

I would like to Thank and Acknowledge the support of: Dr. *Samuel Ippolito*, the Course Coordinator of corresponding “EEET2166 /1262 Real-time Systems Engineering & Design” course from RMIT Melbourne.

Standard Input/Process/Output Model

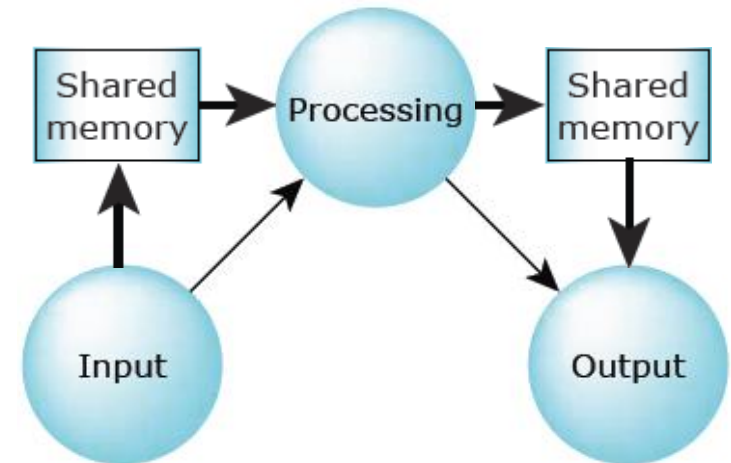
- A generic input/process/output model can be used to separate input, processing and output tasks:
 - **Input** : Get input form somewhere
 - **Process** : Process the input to produce something useful for either output or control
 - **Output** : Using the output or control to do something
- In the case of a **One-thread-per-process design**, we'd have three processes, literally an **input** process, a “**processing**” process, and an **output** process
- Loosely coupled design, where the communication path could be accomplished over some connection protocol
 - Pipes (file access, command line redirection, etc.)
 - POSIX message queues
 - Message passing
- **Advantages**: each processes can be easily recompile/redesign/replaced.
- Can be distributed among multiple nodes
 - very powerful *scalability* factor



Multiple operations, multiple processes [2]

Multiple Processes with Shared Memory

- **Optimized communication path can increase the data flow between processes.**
 - Done by make the coupling between the three processes tighter by requiring each process to have a known method to access a shared resource.
 - Instead of using a general-purpose protocols, shared memory scheme can be used
 - Thick lines indicate data flow,
 - Thin lines indicate control flow (which can use message passing, queues, signals, etc.)
- Tighter design coupling results in **faster and more efficient data flow**
 - Good for situations where the data requires high bandwidth and the control information consumes only a very small amount of bandwidth.
 - Shared memory between processes require each process to map a common region of memory (address space) within the same node.
- **Disadvantages:** lose the ability to distribute over a network as QNX doesn't support shared memory over networks.
 - All processes now need to conform to the shared-memory model (**specific interface**)
- **Advantages:** Still maintain ability to independently recompile/redesign/replaced a process without modifying other processes.



Multiple processes with shared memory [2]

Safety & Liveness

- **Safety**

- Something bad will not happen, for example:
 - Asynchronous concurrent processes will not interfere with one another
- **Safety failure:**
 - Interference can result in one process corrupting another process

- **Liveness**

- Something good will happen, for example:
 - Making sure the system functions correctly with the correct behavior
- **Liveness failure:**
 - One or more processes are unable to continue execution
 - One or more processes are unable to meet their deadlines

- **Mutual Exclusion**

- Only one process is allowed to exclusively access or hold a shared resource at one time.
 - Required for safety
- But must release the resource within a reasonable time
 - Required for Liveness
 - Avoidance of Livelock, Starvation or Deadlock

Test and Set Algorithm

- The test and set method is used to provide mutual exclusion between 2 processes.
 - Requires the use of indivisible test-and-set instructions (atomic operations)
 - Indivisible or atomic operation are typically built into the hardware as specialised instructions for synchronisation purposes.

```
// C like operation - can't be implemented
testandset(&a, &b) ≡ {
    a = b; // xcannotenter = active (0)
    b = 1; // set active to True
} // function executes indivisibly -
    // atomic uninterruptable instruction
```

```
int active = 0; // Process can't enter CS
                // if active == 1;
void * process1( void )
{
    int onecannotenter;
    while(1)
    {
        onecannotenter = 1;
        while(onecannotenter)
            testandset(&onecannotenter, &active);

        criticalsection1;
        active = 0;
        // other process1 code ...;
    }
}
```

```
void * process2( void )
{
    int twocannotenter;
    while(1)
    {
        twocannotenter = 1;
        while(twocannotenter)
            testandset(&twocannotenter, &active);

        criticalsection2;
        active = 0;
        // other process2 code ...;
    }
}
```

Only the code required to enforce mutual exclusion is shown

Test-and-Set Instruction

- The test-and-set CPU instruction is used to implement mutual exclusion in multiprocessor environments.
 - Can be used to provide mutual exclusion for more than 2 processes.
 - Does not require any special OS support, rather it
 - Requires the support of a special instruction (hardware)
- Typically, the test-and-set operation **cannot** be implemented in software alone.
 - The software loop needs to invoke a special test-and-set operation which will run atomically (i.e. Not be interrupted or preempted).
 - Typically implemented as an atomic *read, modify, write* instruction which takes two operands
 - Most CPUs have some assembly instructions which are guaranteed atomic operations, such as XCHG on x86 processors and load-link/store-conditional on Alpha, MIPS, PowerPC, and other architectures.
 - These instructions are intended to provide a way to build synchronization primitives more efficiently than can be done with pure shared memory approaches.
- Multi-core systems need to provide a **bus arbitration mechanism** to limit access to shared memory so only 1 CPU core at a time has access.
 - The lock can lead to resource contention, resulting in livelock caused by:
 - The address/data bus locking up, and/or
 - cache invalidation, when test-and-set operation needs to access memory atomically.

Test and Set Algorithm

- In order to ensure that the Test and Set Algorithm works, we need to ensure that the thread can enter the Critical Section (CS) under the minimum set of cases:
 1. Only 1 process is active: Process 1 wants to enter CS (simplest case)
 2. Both processes are active: Process 1 in CS and Process 2 wants to enter CS
 3. Both processes are active: Process 1 and Process 2 want to enter CS at exactly the same time (ie multi-core CPU system)
 - The fact that the *test-and-set* instruction executes as one instruction means that the race condition in Peterson's Algorithm is practically avoided.
 - However, the address/data bus could become locked down, which could lead to a situation of **Indefinite Postponement** due to one process hogging a resource.
 - Therefore the test and set algorithm (as shown here) can't be considered a complete solution.
-
- Crude ways to implement your own Mutual exclusion on a Single core CPU without employing the discussed techniques can be achieved:
 - If the CS is really small and your shared data is only used by 1 thread and ISRs, you can disable interrupts within your program so you can guarantee that the ISR don't effect your shared data during the CS within the thread.
 - If you have multiple threads using shared data you could disable time slicing (use FIFO scheduling) to guarantee your thread finishes its work before starting a new thread.